

GITHUB COPILOT

Facilitator Session Guide

Duration: 60 Minutes · Audience: Developers, BSAs & Product Owners
Toronto, Ontario · 2026

How to Use This Guide

This document is your complete facilitator playbook for a 60-minute interactive session on GitHub Copilot with Company co-op students. Each section maps to a time block on the agenda and contains everything you need: what to say, what to show, what questions to ask, and what to watch out for.

You do not need to read this verbatim. Use it to prepare your talking points before the session, and keep it open on a second screen as a reference during delivery.



Before you start — facilitator checklist

- Test your Copilot access in VS Code — confirm Chat and inline autocomplete both work
- Have a simple TypeScript or JavaScript file open and ready for the live demo
- Know the Company Copilot policy: what can co-ops paste in? (no client data, no secrets)
- Prepare a short URL or QR code to share the cheat sheet at the end
- Arrive 10 minutes early to test screen sharing and font size on the projected display

Session Overview

Your at-a-glance reference. Use this to track pace throughout the session.

Time	Segment	Duration	Type
0:00–0:05	Opening & Icebreaker	5 min	Anchor
0:05–0:13	AI, LLMs & Models — the mental model	8 min	Must Cover
0:13–0:23	Prompt Engineering — the core skill	10 min	Must Cover
0:23–0:30	Exercise #1 — Prompt Makeover	7 min	Must Cover
0:30–0:35	Token Usage & Context Windows	5 min	Good to Cover
0:35–0:43	Copilot Features Deep Dive	8 min	Must Cover
0:43–0:47	MCP Tools — Conceptual Intro	4 min	Good to Cover
0:47–0:55	Exercise #2 — Copilot Live	8 min	Must Cover
0:55–1:00	Wrap-up & Q&A	5 min	Anchor

Colour key: **Must Cover** = essential, never skip **Good to Cover** = include if on pace **Anchor** = transition block

Purpose

Set the energy for the room. Establish that this is interactive and conversational, not a lecture. Get a quick read on the group's existing AI experience so you can calibrate depth on the fly.

Opening — What to Say

Introduce yourself in one sentence: your role, your team, and one thing about how you use Copilot day-to-day. Then immediately invite participation:

"Before we get into the content — quick show of hands. Who here has used GitHub Copilot, ChatGPT, or any AI coding tool in the last month?"

Pause. If fewer than half raise their hands, you may need to slow down on theory. If most are up, you can move faster. Then ask:

"Second question — if you could leave today knowing how to do one thing better with AI tools, what would that be?"

Take 2–3 verbal answers. Write them down visibly. This signals the session is two-way and gives you real-time calibration data.

Agenda Walkthrough — 60 Seconds

- We'll build a quick mental model of what AI and LLMs actually are — no math, just enough to understand why prompts matter
- We'll spend the biggest chunk on prompt engineering — the single most transferable skill from today
- We'll do two short exercises — not watching, actually doing
- We'll cover the Copilot features you have access to right now
- Last 15 minutes are Q&A — no question is too basic

Ground Rules — Say These Out Loud

- This is interactive — ask questions now, not later
- There are no dumb questions. Even experienced developers are still figuring this out
- If something doesn't match what you see in your editor, tell me — environments differ
- Open your laptop and follow along if you want. Hands-on beats watching



Facilitator tip

- If the room is quiet, ask one person directly: "What team are you on, and have you tried Copilot yet?" — a direct question breaks the ice faster than a general one.

Purpose

This is not a deep AI lecture. The goal is a sufficient mental model — enough to understand why prompt quality matters and why the tool sometimes gets things wrong. Every concept here connects directly to something they will experience with Copilot today.

The Funnel: AI → ML → LLM

Narrate this exactly as a funnel — draw it if you have a whiteboard:

"Artificial Intelligence is the broad field — any system that appears to do intelligent things. Machine Learning is a subset: systems that learn from data rather than following hand-coded rules. Large Language Models are a specific type of ML system, trained on enormous amounts of text."

"GitHub Copilot is powered by an LLM from OpenAI — a version of the GPT-4 family, fine-tuned on public code repositories. When you type in your editor, the model sees your code as text and predicts what text should come next."

The Core Concept: Next-Token Prediction

This is the single most important idea in this segment. Make sure it lands before moving on:

"An LLM is not a search engine. It's not looking up answers in a database. It has learned, from trillions of examples, what text tends to follow other text. When it writes code, it is generating the statistically most likely continuation of what you've written so far. It's very sophisticated autocomplete."

Connect it immediately to why this matters:

- It explains hallucinations: the model generates plausible-looking text, not guaranteed-correct text
- It explains why context matters: the more relevant context you give it, the better the prediction
- It explains why vague prompts give vague results: not enough context to constrain the prediction

Models: Who Makes What

Model	Creator	Used In	Notes
GPT-4o	OpenAI	GitHub Copilot	Powers Copilot at Company today
Claude 3.5	Anthropic	Claude.ai, Cursor	Strong at reasoning and writing

Gemini	Google	Google Workspace AI	Integrated with Docs and Sheets
Llama 3	Meta	Self-hosted deployments	Open weights — can run privately

Hallucinations — Say This Explicitly

"Here is the most important safety rule for using any AI tool: the model does not know when it is wrong. It generates an incorrect answer with exactly the same confidence as a correct one. This is called a hallucination. It is not a bug — it is a fundamental property of how these systems work."

Concrete examples to share:

- Copilot suggesting a function that does not exist in a library
- Copilot generating code that looks right but has a subtle off-by-one error
- Copilot referencing a file path that does not exist in your project

"The rule is: always verify. Copilot is a very fast first draft, not a source of truth. Your code review skills and your test suite are still your safety net."



Company Copilot policy — say this before moving on

- Copilot is approved for use at Company for code generation and development tasks
- Do NOT paste client data, account numbers, customer PII, or internal security credentials into Copilot Chat
- Code you write with Copilot's help is still your code — you are responsible for reviewing it before committing
- If unsure whether something is safe to paste, don't paste it — ask your manager or check with InfoSec
- Copilot sends your prompt to external servers — treat it like sending a message to a smart colleague outside the building

Purpose

This is the highest-value segment in the session. The skill of writing good prompts is immediately useful today, applies equally to all three roles in the room, and directly predicts how much value someone gets from Copilot. Do not rush this section.

What Is a Prompt?

"Everything you type to Copilot is a prompt. The code above your cursor is a prompt. The comment you write before a function is a prompt. The question you type in Copilot Chat is a prompt. The model only knows what you tell it."

The 4-Part Prompt Formula

Write this on the whiteboard. It applies to all AI tools, not just Copilot.

Component	What It Does	Example
Role	Tell Copilot who it is	You are a senior TypeScript developer...
Context	Give the relevant background	...working on an Express REST API...
Task	Describe exactly what you need	...write a JWT validation middleware...
Constraint	Define format, length, limits	...return 401 JSON error. No extra libraries.

Walk through each component, then combine them into one full example:

"You are a senior TypeScript developer working on an Express REST API. Write a middleware that validates JWT tokens in the Authorization header. Return a 401 with a JSON error object { error: 'Unauthorized' } if the token is missing or invalid. Do not use any third-party JWT library other than jsonwebtoken."

Compare that to what most people actually type: "write JWT middleware"

Ask the room: "Which one produces better code?" — rhetorical, but it drives the point home.

Bad Prompt vs Good Prompt — Three Examples

Developer

✗ **Bad:** *fix this bug*

✓ **Good:** *The following TypeScript function should return the total price including 13% HST. It is returning NaN for some inputs. Identify the bug, explain why it happens, and return a corrected version with a comment explaining the fix.*

Business Systems Analyst

✗ **Bad:** *write requirements for login*

✓ **Good:** *You are a BSA at a Canadian financial institution. Write functional requirements for a multi-factor authentication login flow for a wealth management web app. Include the happy path and two failure scenarios. Format as numbered acceptance criteria using Given/When/Then.*

Product Owner

✗ **Bad:** *write a user story*

✓ **Good:** *You are a product owner at a wealth management company. Write a user story for a client who wants to view their portfolio performance over the last 12 months on a mobile device. Include the persona, goal, benefit, and three acceptance criteria. Use 'As a... I want... So that...' format.*

Iteration: Prompting Is a Conversation

"The most common mistake is writing one prompt and giving up if the output is not perfect. Copilot Chat is a conversation — you should iterate."

- First pass: generate a rough draft
- Second pass: 'Make the error handling more specific — use a custom `AppError` class'
- Third pass: 'Now add JSDoc comments for each parameter'
- Fourth pass: 'Refactor this to use `async/await` instead of `.then()` chaining'



Common reasons Copilot goes wrong — and the fix

- No technology specified: it picks a language or framework you did not intend. Fix: always name your tech stack
- No error handling requirement: it generates the happy path only. Fix: explicitly ask for error cases
- No format constraint: it gives you the wrong structure. Fix: specify the output format you need
- Too much context in one prompt: it loses track of requirements. Fix: break into smaller, sequential prompts
- Outdated library API: it suggests functions from an older version. Fix: specify the version or paste the API signature

Purpose

The first hands-on moment. Students rewrite a weak prompt into a strong one using the 4-part formula. This is role-specific so every person in the room has a task relevant to their actual work.

Instructions — Read These Out

"I'm going to put a bad prompt on the screen. Using the 4-part formula — role, context, task, constraint — rewrite it into a better one. You have 3 minutes. Work alone or with the person next to you. Then we'll share a few."

The Prompts

Role	The bad prompt to rewrite
Developer	"Write a function to get user data"
BSA	"Document the payment process"
Product Owner	"Write a story for the dashboard"

Sample Answers — For Facilitator Reference

Developer — improved

✗ **Bad:** "Write a function to get user data"

✓ **Good:** "You are a senior Node.js developer. Write a TypeScript async function called `getUserById` that accepts a `userId` string, queries a PostgreSQL database using the `pg` library, and returns a typed `User` object. If the user is not found, throw a custom `NotFoundError`. Include a JSDoc comment."

BSA — improved

✗ **Bad:** "Document the payment process"

✓ **Good:** "You are a BSA. Document the end-to-end payment processing flow for an online wealth management platform. Include: the actors involved, the trigger event, each step in sequence, validation checks, and two failure paths (insufficient funds, declined card). Format as a numbered process flow."

PO — improved

✗ **Bad:** "Write a story for the dashboard"

✓ **Good:** "You are a product owner at an investment platform. Write a user story for a retail investor who wants to see a real-time summary of their portfolio value on the main dashboard. Include persona, goal, business benefit, and three acceptance criteria covering data freshness, mobile responsiveness, and accessibility."

Debrief — 2 Minutes

Call on 2–3 people to share their rewrites. For each one ask:

- "What did you add that was not in the original?"
- "What constraint did you include and why?"
- "Would you trust Copilot's output more with your version than the original?"

Close with: "The skill you just practised — that is the whole game. Everything else in AI tooling builds on this."

Purpose

'Good to Cover' — include if you are on pace. If running 2+ minutes behind, condense to the key practical points only. The goal is practical understanding, not tokenization theory.

What Is a Token?

"A token is roughly a chunk of text — about 4 characters or three-quarters of a word on average. The sentence 'Write me a function' is about 5 tokens. A 1,000-line file might be 10,000 tokens."

"Why does this matter? Two reasons: the model has a limit on how many tokens it can process at once — the context window — and in commercial use, you are often billed by token consumption."

Context Window — The Model's Working Memory

"Think of the context window as the model's short-term memory. It can only 'see' what fits inside it. Everything outside the window is invisible to the model."

Practical consequences:

- Long conversations drift: the model forgets what you said at the start of the chat
- Pasting a huge file: if the file exceeds the context window, it gets truncated — silently, with no warning
- Multiple open files: Copilot uses the active file and some neighbouring context, not your whole codebase
- Bloated prompts: 500 words of context leave less room for the model's output



Token-smart habits — for all roles

- Start a new chat for each new topic — do not carry stale context from a different task
- Be concise: every extra word in your prompt consumes tokens that could go toward the response
- Paste only the relevant section of a large document, not the whole thing
- If the response gets cut off mid-sentence: the model hit its output limit — ask it to continue
- For POs and BSAs: do not paste a 50-page requirements doc expecting a useful summary — extract the relevant 2 pages first

Purpose

Map the actual tools students have access to right now. Show each feature in the context of a real task. Most people only know inline autocomplete — this section dramatically expands what they will try in the coming weeks.

Feature 1: Inline Autocomplete

"This is the ghost text you see in your editor as you type. Copilot watches your code in real time and suggests what comes next."

Demo live:

- Type a function signature in TypeScript — watch the body appear as ghost text
- Press Tab to accept the full suggestion
- Press Ctrl+Right (Windows) or Option+Right (Mac) to accept suggestion word by word
- Press Escape to dismiss
- Write a comment describing intent before the function — show how it dramatically improves the suggestion

"The single best trick for inline autocomplete: write your intent as a comment first. Something like '// Validate Canadian phone number, return boolean' — then Copilot writes the function body for you."

Feature 2: Copilot Chat & Slash Commands

"Copilot Chat is a conversational interface. It can see your current file, your selected code, and your conversation history. You ask it questions about your code — or ask it to do things to your code."

Show each slash command live:

Command	What It Does
<code>/explain</code>	Explains selected code in plain English
<code>/tests</code>	Generates unit tests for selected code
<code>/fix</code>	Identifies and fixes bugs in selected code
<code>/docs</code>	Generates JSDoc inline documentation
<code>/new</code>	Starts a fresh chat session (clears context)

"For BSAs and POs: Copilot Chat is where you will spend most of your time. You do not need to be in a code editor. You can ask it to help you write BDD scenarios, analyse requirements gaps, draft user stories, or explain what a piece of code does in plain English."

Feature 3: Custom Instructions

"You can give Copilot standing instructions that persist across all your chats. For example: 'Always use TypeScript strict mode. Prefer functional patterns over class-based ones. When writing tests, use Jest and follow the Arrange-Act-Assert pattern.' Copilot will apply these automatically without you having to repeat them."

Where to set it: VS Code Settings → Copilot → Custom Instructions.



Quick wins for each role

- Developers: use `/tests` immediately after writing a function — instant test coverage bootstrap
- BSAs: paste a user journey description in Chat and ask `/explain` what edge cases are missing from this flow'
- POs: type your raw feature idea in Chat and ask it to 'rewrite this as a well-structured user story with acceptance criteria'
- Everyone: use `/explain` on any unfamiliar code section — faster than Stack Overflow for basic comprehension

Purpose

Plant the seed for where AI tooling is going. This is conceptual only — no live demo, no deep technical detail. The goal: students leave knowing the term MCP, understanding what it enables, and being curious enough to explore it on their own.

What Is MCP?

"MCP stands for Model Context Protocol. It is a standard that lets an AI model connect to external tools. Think of it as giving the model hands."

Without MCP, an LLM can only work with what you paste into the chat. With MCP, the model can:

- Read files from your filesystem
- Query a database directly
- Call an API — GitHub issues, Jira tickets, Confluence pages
- Run a terminal command
- Search the web in real time

"The model goes from being a very smart notepad to being an agent that can actually do things on your behalf."

Copilot Extensions — The Practical Equivalent

"Copilot has its own version of this called Copilot Extensions. A few real examples:"

- Copilot + GitHub Issues: the model reads your open issues, understands the codebase, and suggests where to make the fix
- Copilot + Jira: pull a ticket number into Chat and get implementation suggestions based on the acceptance criteria
- Copilot + your internal documentation: ask questions and get answers grounded in your actual team docs

"This is early-stage at most organizations, but it is moving fast. Knowing what MCP is puts you ahead of most developers right now."



If MCP questions come up

- Offer a follow-up session focused on MCP and agents — this topic deserves its own 60 minutes
- Point students to modelcontextprotocol.io for self-study
- Tools like Cursor and Claude already support full MCP integrations — worth exploring outside of the Company environment

Purpose

Students actually use Copilot — not just think about it. The task is role-specific and short enough to complete in 5 minutes, with 3 minutes for sharing. The goal is a tangible first win they can build on after the session ends.

Instructions — Read These Out

"Open your editor and Copilot Chat. I am going to give you a task based on your role. You have 5 minutes. Use the 4-part formula we practised. If your first output is not great, iterate — do not give up after one try."

The Tasks

Role	Your task
Developer	Generate a TypeScript utility function that validates a Canadian phone number (10 digits, with or without country code +1). Then use /tests to generate unit tests for it. Then use /explain to have Copilot explain how the regex works in plain English. Three Copilot interactions in 5 minutes.
BSA	Paste the following into Copilot Chat: 'A client can transfer funds between their own accounts.' Ask Copilot to expand this into a full requirements specification including: pre-conditions, trigger event, main success scenario, two alternative flows (insufficient funds and daily limit exceeded), and post-conditions. Format as a structured use case.
Product Owner	Ask Copilot to write a complete user story for: 'A mobile user wants to set up biometric login for faster access to their investment account.' Include persona, goal, benefit, and five acceptance criteria. Then in a follow-up prompt, ask it to split this into two smaller, sprint-sized stories if the acceptance criteria are too broad.

What to Watch For

- Students who get a bad first output and give up immediately — prompt them to iterate: 'What was missing from your prompt?'
- Students who get a good output but do not know how to evaluate it — remind them to verify, not blindly trust
- Students who finish early — ask them to write the worst possible prompt for the same task and compare outputs side by side

Debrief — 2 to 3 Minutes

Pick 2 people to share their screen and walk through their prompt and output. For each:

- "What prompt did you use?"
- "What did you like about the output?"
- "What would you change on the next iteration?"

Close with: "You just saved at least 10 minutes of boilerplate work. That is the daily value proposition of this tool — not replacing you, just removing the tedious first-draft work so you can focus on what actually requires your judgement."

Purpose

Consolidate the session into 3 memorable takeaways, equip students with one concrete action for the week, share the cheat sheet, and open the floor for questions.

The 3-Point Recap — Say These Exactly

- "Prompts matter. The quality of your output is directly proportional to the quality of your input. Role, context, task, constraint."
- "Context is limited. The model only knows what you give it. Be intentional, start fresh chats for new topics, and always verify outputs."
- "Copilot is a collaborator, not an oracle. It speeds up the work. You still own the decision, the review, and the result."

What to Try This Week

"Pick one task you would normally do by muscle memory this week. A function you have written before, a document you have drafted before, a user story in your usual format. Do it with Copilot first. Notice where it helps and where it falls short. That is your learning for the week."

Share the Cheat Sheet

Share your short URL or QR code here. The cheat sheet should include:

- The 4-part prompt formula: Role + Context + Task + Constraint
- Copilot slash commands: /explain, /tests, /fix, /docs, /new
- The 3 rules: be specific, iterate, always verify
- Data safety reminders for Company environment
- Where to get more help: Copilot documentation, your team's internal AI community channel

Open Q&A

"The floor is yours — any question is fair game. If I do not know the answer, I will say so and we will find out together or I will follow up async."



Common Q&A questions to prepare for

- Can Copilot see my entire codebase? → Only the current file and some neighbouring context, not your whole repo
- Is Copilot training on my code? → With enterprise plans, code is not used for model training. Verify the exact Company policy with your manager.
- What is the difference between Copilot and ChatGPT? → Same underlying model family, different integrations. Copilot is code-focused and IDE-integrated. ChatGPT is general-purpose via a browser.
- Should I always use Copilot? → Use it where it saves time. For complex architecture, security-critical code, or tasks where deep understanding matters, manual work is still the right call.
- How do I make Copilot work better with our team conventions? → Use custom instructions or add a COPILOT.md file in your repo to give it standing context about your stack and patterns.

Appendix: Quick Reference Cheat Sheet

Print this page or share it as a standalone PDF for students to keep after the session.

The 4-Part Prompt Formula

Component	What It Does	Example
Role	Tell Copilot who it is	You are a senior TypeScript developer...
Context	Give the relevant background	...working on an Express REST API...
Task	Describe exactly what you need	...write a JWT validation middleware...
Constraint	Define format, length, limits	...return 401 JSON error. No extra libraries.

Copilot Slash Commands

Command	What It Does
<code>/explain</code>	Explains selected code in plain English
<code>/tests</code>	Generates unit tests for selected code
<code>/fix</code>	Identifies and fixes bugs in selected code
<code>/docs</code>	Generates JSDoc inline documentation
<code>/new</code>	Starts a fresh chat session (clears context)

Data Safety at Company



What NOT to paste into Copilot

- Client names, account numbers, SIN numbers, or any personally identifiable information
- Internal security credentials, API keys, or passwords
- Unpublished financial data or proprietary trading strategies
- Full production database dumps or query results containing client records
- If in doubt: ask your manager. The 10-second check is always worth it.